

Contents

romp simplification inventory: where the complexity is and what to remove	1
Summary	1
The five findings	2
Finding 1: one behavior, two implementations	2
Finding 2: the request data model is overgrown, and its size sets everything downstream	2
Finding 3: the judge is fought, not trusted	2
Finding 4: whole features can be removed without touching the spine	3
Finding 5: time-based workarounds persist against the project's own rule	3
Also worth removing	3
Decisions that set the order of work	3
Reference	4
Architecture	4
Components	4
Producer pipeline (Python, bin/)	4
Read-side terminal views (Python, bin/)	4
Session lifecycle and mail (Python/bash, bin/ and hooks/)	4
Kernel (TypeScript, chat-view/src/kernel/)	5
Shared TypeScript modules (chat-view/src/)	5
Obsidian views (obsidian/)	5
Records on disk	5
summaries/<fsid>.jsonl	6
requests/nodes.jsonl	6
requests/links.jsonl	6
requests/cleared.jsonl	6
requests/followups.jsonl	6
requests/corrections.jsonl	6
requests/decision-log.jsonl	6
Other records	7
What the data model shows	7
The fold rules	7
The model roles	8
Where code is duplicated	8
Deletable features, with line counts	8

romp simplification inventory: where the complexity is and what to remove

Built 2026-06-13. Line numbers were gathered in a mapping pass; verify them at edit time.

Summary

romp runs every day, and it carries far more code than its behavior needs. The system is about 24,000 lines: 8,000 of Python in bin/, 13,200 of TypeScript in chat-view/, and 3,200 of JavaScript in obsidian/. The goal is to keep every feature while making the logic and data structures much simpler, almost entirely by deleting code rather than adding it. This document maps every component, every on-disk record, and every concentration of complexity, so the removal work has one shared reference to run against.

Five findings account for most of the removable complexity, and they compound:

1. The read-side fold and the transcript parser are each written twice, once in Python and once in TypeScript, and kept in step by hand.
2. The request data model carries roughly twice the node kinds, tags, and fold rules its behavior needs.

3. The pipeline does not trust its own model: it wraps the judge in deterministic patches, a regex override, and a second pass that overrules it.
4. About 1,500 lines sit in self-contained features that can be deleted whole, several of which the code already flags as low-value or impossible.
5. Several components guess session state from clocks and screen-scrapes because no event reports it, which the project's own design rule forbids.

The request data model is the lever. It sets the size of the judge's prompt, the producer that writes the records, and both copies of the fold that reads them, so shrinking it shrinks all of them at once. Most of what romp does can be expressed in far less code, and this is the change that moves the rest.

The findings below make the argument. The decisions section lists what only you can settle, because those answers set the order of work. The reference half holds the component inventory, the record schemas, the fold-rule table, the model roles, the duplication index, and the deletion checklist, for lookup rather than linear reading.

The five findings

Finding 1: one behavior, two implementations

The read-side fold and the transcript parser are each implemented twice and synced by hand. `bin/romp-feed` computes the ask columns in `ask_items/feed_items` (about 390 lines of Python); `chat-view/src/kernel/feed.ts` computes the same columns in `computeAskItems/computeFeedItems` (about 620 lines of TypeScript). The two carry the same node-status logic, the same rollup, the same column derivation, the same liveness and claim-lag rules, down to cross-referencing comments and shared incident dates. A parity test (`kernel/fold-parity.test.ts`) exists only to catch them drifting apart. A third partial copy lives in `romp-pipeline's _disposition`, and the recency colormap is duplicated a further three times.

The transcript parser repeats the pattern. `bin/romp-events` (696 lines) and `chat-view/src/transcript.ts` (531 lines) both walk the same files with the same compaction stitch, active-path reconstruction, and queue fold, then emit different shapes. The kernel reads `romp-events'` cached output through `state.ts` while also parsing the same transcripts itself through `transcript.ts`, so it pays for the parse twice.

The removal is to pick one language per concern and have the other side call it. The kernel can shell out to `romp-feed --json`, or both sides can generate from one spec. Collapsing the fold deletes one full implementation (400 to 620 lines) plus the parity test plus the standing cost of keeping two copies aligned; collapsing the parser can remove a further 500. This is the highest-leverage move in the document, and it gets easier once Finding 2 has shrunk the rules.

Finding 2: the request data model is overgrown, and its size sets everything downstream

The request data model carries more kinds, tags, and rules than its behavior needs, and every extra one multiplies across the producer and both consumers. There are 5 node kinds, 6 relevance tags, and about 21 fold rules. Roughly half the rules are patches added one incident at a time: follow-up resurrection (which the spec itself expects to "wither and die"), answer-rows-as-relinks (which shortcut an inference that still runs as the fallback), the WAIT and IDEA exemptions, the claim-lag hold, the blind-read guard. Of the 6 tags, only three drive distinct behavior; WAIT and DETAILS both mean "not actionable," and the writer role models only three tags rather than six.

The removal is to collapse the tags toward `{done, needs-user, routine}` with a single flag for what clears a card, drop the answer node kind in favor of its existing fallback, retire follow-up resurrection once every-prompt-mints-an-ask is trusted, and fold WAIT into routine for routing. Each deletion removes one branch from `bin/romp-feed`, the same branch from `kernel/feed.ts`, and the matching paragraph from the judge's reply prompt. The fold-rule table in the reference half marks each rule core or patch, so it doubles as the worklist for this finding.

Finding 3: the judge is fought, not trusted

The pipeline distrusts its own model's output, so it wraps the judge in deterministic code that overrules it. `_handle_rep` in `romp-summarize-backfill` layers four separate link-injection mechanisms on top of the model's LINK field: an own-turn link, an amended-or-answered link, a run-aware link, and a DONE default. A regex,

`_tail_question`, overrides the model's tag when the reply ends in a question. A second and stronger model, the Opus brief auditor, re-judges DECISION links and writes corrections that demote them, 38 times in the live log. All of this exists because the LINK/DONE/TAG output is unreliable.

The design's own note points at the fix: attach a reply to its own turn's ask structurally and always, which lets the reply call shrink to detecting DONE and nothing else. Doing so deletes most of `_handle_rep`, the regex override, and the demotion auditor, removing 50 to 80 lines of patch code, one Opus role, and a section of the largest prompt in the pipeline.

Finding 4: whole features can be removed without touching the spine

About 1,500 lines sit in coherent features that can be deleted outright, and several are already flagged in their own comments. The missed-handoff suspect auditor admits "near-zero precision, nearly all FYIs." The `--rejudge` path is dead one-time backfill keyed to a hardcoded date. The two non-tmux backends (`headless` and `stream`, 527 lines) are non-default, and `stream` supersedes `headless`. In the postal service, the immediate-wake injection (about 370 lines of prompt-box reverse-engineering) plus the retry, wake, and orphan-sweep loops can go, because the Stop-hook drain is the documented sufficient backstop. The session-search behind `find_sessions` (about 130 lines) serves one MCP tool. The postal HTTP bus exists only for the remote-tunnel feature; a local-only design could use a file lock instead. The blocked-session card in the feed describes itself as "NOT a supported steady-state card type."

These removals are independent of the spine, so they are the low-risk way to clear noise before the data-model work. The deletion checklist in the reference half lists each one with its line range and rough line count.

Finding 5: time-based workarounds persist against the project's own rule

Several components infer session state from clocks and screen-scrapes because no event tells them the truth, which CLAUDE.md's "event-based over time heuristics" rule forbids. Two of them exist only because Claude fires no hook when a turn is interrupted with Esc: `bin/romp-idle-dots` captures the tmux pane and pattern-matches it to guess whether a "working" session is actually stuck, and `bin/romp-interrupt-reset` does the same guess a second way. Around them sits a swarm of magic-second constants in `romp-pipeline`, `romp-dashboard`, `romp-postal`, and the kernel (`PENDING_WINDOW`, `STALE_AFTER`, `RETRY_INTERVAL`, `ORPHAN_GRACE`, `now-addedAt>6000`, `probeTick%4`, and others).

The honest fix is the one the design note already names: emit a single real interrupt event. With that event in place, both scrapers delete (about 273 lines), and the kernel's interrupt timers re-key onto it.

Also worth removing

Four smaller items round out the worklist. The turn chopper slices one physical turn into as many as five boundary events, then the backfill stitches them back together for link attachment (`RUN_FOLD_KINDS`, `_run_prefix_ids`); the round trip may be avoidable. `server.ts` holds about 350 lines of inline page HTML and CSS that belong in static assets. The Obsidian timeline's `draw()` is a single 515-line function that rebuilds all SVG on every poll, and the perceptual-fade algorithm and the model/effort choice lists are each duplicated across three files. The feed renderer re-derives data-model structure (group-card membership, the lowest open question) at render time, which the "views mirror the data model" rule says belongs in the kernel.

Decisions that set the order of work

Eight questions gate the larger moves, and the answers change what gets built. Each is yours to settle.

1. **Which language wins the fold and the parser?** Python keeps the terminal views working standalone; TypeScript keeps the logic next to the UI host. This decides Finding 1.
2. **Keep any non-tmux backend?** If headless hosting is a real future, keep `stream` and delete `headless`; if not, delete both and remove every `tui:null` branch.
3. **Is `find_sessions` actually used?** If rarely, it is the most self-contained 130 lines to cut.
4. **Keep cross-machine postal?** It is the only thing that needs the HTTP bus and heartbeats; dropping it allows a file-lock store.

5. **Is the turn-boundary drain enough, or is immediate mail wake needed?** Dropping immediate wake removes about 470 lines of pane reverse-engineering and polling.
6. **How many relevance tags?** {done, needs-user, routine} plus a clears-on flag, versus the current six. This sizes the judge prompt and both folds.
7. **Keep the cosmetic tmux decorations?** The peer chips and message pill are about 260 lines of pure display, including the launcher's status-format string.
8. **Source a real interrupt event?** Adding one small hook or binding that emits an event is the price of deleting the two Esc scrapers and the timers around them.

Reference

The sections below are for lookup, not linear reading.

Architecture

Claude Code sessions run in tmux panes, and their transcripts under `~/ .claude/projects/*.jsonl` are the source of truth. The system splits into two halves joined only by the records under `~/ .local/state/romp/`.

The producer half is the Python in `bin/`. `romp-events` chops each transcript into turn events with stable ids. `romp-summarize-backfill`, an always-on daemon, runs four model roles (judge, announcer, auditor, writer) over those events and writes the records. `romp-postal` is the inter-session mail bus.

The consumer half folds the records into surfaces: the feed, the ask columns, the chat, and the timeline. The TypeScript kernel under `chat-view/src/kernel/`, run by `romp-serve`, reads the records and the transcripts, drives sessions through a `SessionBackend`, and serves the UI bundles over HTTP and WebSocket to thin clients in the browser, the VS Code extension, and Obsidian.

Components

Producer pipeline (Python, `bin/`)

File	LoC	Role
<code>romp-summarize-backfill</code>	1718	The daemon. Judge (request/reply/message) and auditor (brief/suspect). Sole writer of records.
<code>romp-events</code>	696	Deterministic turn chopper: 5 boundary kinds, stable ids, active-path walk, compactification.
<code>romp-digest</code>	158	Writer role: rolling per-session profile to <code>digest/<sid>.json</code> .
<code>romp-feed-detail</code>	224	Writer role: per-card expand paragraph to <code>feed-detail/<id>.json</code> .
<code>romp_colormap.py</code>	34	Shared recency colormap (<code>age_rgb</code>). Pure.

Read-side terminal views (Python, `bin/`)

File	LoC	Role
<code>romp-feed</code>	688	Terminal feed. The ask fold in Python (<code>ask_items/feed_items</code>), twin of <code>kernel/feed.ts</code> .
<code>romp-pipeline</code>	708	Per-turn pipeline-health table and trace pane. Third partial fold (<code>_disposition</code>).
<code>romp-ledger</code>	227	Per-session digest view.
<code>romp-dashboard</code>	396	Terminal dashboard (bash); polls tmux vars only.
<code>romp-idle-dots</code>	212	Idle-dot fade and the stuck-working healer (<code>screen-scrapes</code> panes).

Session lifecycle and mail (Python/bash, `bin/` and `hooks/`)

File	LoC	Role
romp	829	Launcher: spawn/resume/attach, identity colors, self-pr
romp-postal	1776	Postal bus, MCP server (8 tools), CLI, hook helpers, tmux
romp-manager	205	Supervises the kernel (romp-serve): respawn and cont
romp-interrupt-reset	61	Esc-interrupt workaround (no hook fires); pane-scrape h
romp-mail-clear	16	Clears mail-badger tmux vars.
romp-serve	35	Builds (if stale) and runs chat-view/dist/kernel.js
romp-timeline-serve	66	Standalone timeline server, largely superseded by kerne
hooks/tmux-status.sh	~148	State machine: hook event to state; writes states/<si
hooks/romp-summarize.sh	~344	The announcer: live Haiku phrase to a tmux var; keeps
hooks/romp-postal-{drain,ensure,revive}.sh	~46/28/60	Mail delivery at turn boundary, bus ensure, parked-mail

Kernel (TypeScript, chat-view/src/kernel/)

File	LoC	Role
server.ts	1893	HTTP+WS hub, message routing (~58 up, ~33 down), poll tick, feed recompute, page build
feed.ts	620	The ask fold in TypeScript, twin of bin/romp-feed.
chat.ts	402	Session mirroring, chip state, AskDriver (live picker driving).
state.ts	441	Pure record-file readers and writers.
backend.ts	75	The SessionBackend interface (the seam).
tmux-backend.ts	225	The real, default backend. Full tui.
headless-backend.ts	265	claude -p per turn. tui=null. Non-default.
stream-backend.ts	262	Long-lived stream-json process. tui=null. Supersedes headless.
timeline.ts	165	/timeline page; require()s the Obsidian JS live.

Shared TypeScript modules (chat-view/src/)

File	LoC	Role
webview/render.ts	2703	Chat-panel renderer.
webview/feed.ts	1771	Feed-panel renderer.
webview/time-marker.ts	65	Rail time-marker labels (DOM-free, tested).
transcript.ts	531	Incremental transcript parser. Overlaps bin/romp-ev
askparse.ts	278	Picker-screen parser (irreducibly complex; parses an
postal-spec.ts	223	Folds peer-mail out of transcript prose into cards.
extension.ts	502	VS Code thin client: spawn-or-attach kernel, host tw
quote.ts / page-skeleton.ts / kernel-attach.ts	27 / 37 / 41	Small helpers; kernel-attach is front-end logic tha

Obsidian views (obsidian/)

File	LoC	Role
romp-timeline-view.js	1677	Timeline SVG renderer; one 515-line draw().
romp-timeline-data.js	667	Timeline data assembler (runs romp-events --emit).
romp-dashboard.js	421	Live session-status view (polls tmux).
romp-logic.js	266	Pure dashboard helpers and color math.
romp-kernel-views.js	165	Iframe wrapper mounting kernel pages as Obsidian leaves.

Records on disk

These records under `~/local/state/romp/` are the only interface between the two halves. Everything else produces or folds them, which makes this the floor of the rebuild. Schemas are as observed in code; the flag marks each as load-bearing, legacy, or patch.

summaries/<fsid>.jsonl

Writer: backfill. Read by everyone.

- request: {id, t, kind:"request", text}
- reply: {id, t, kind:"reply", text, relevance}, relevance in DONE, DECISION, ACTION, IDEA, WAIT, DETAILS

id is the romp-events event id (<fsid>:<turnStartEpoch>:<sha1[:8]>) and is the real key; t is kept only for legacy time-window binding. Keyed by fork fsid, not anchor sid. 11 of 69 live files are id-less pre-migration legacy.

requests/nodes.jsonl

Writer: backfill only. Folded by _load_registry (backfill:347) and ask_items (romp-feed:215).

- {kind:"ask", id:"<turn_eid>#<i>", sid, turn_id, t, text}: a user ask (root)
- {kind:"internal", id:"<msg_id>", from_sid, to_sid, t, text}: an agent-to-agent handoff; node id is the postal message id
- {kind:"parents", id, parent_ids:[...], t}: DAG edges, a separate record so they can land after the node
- {kind:"amend", id:"<ask id>", turn_id, t, text}: fold takes the latest text
- {kind:"answer", id:"<ask id>", turn_id, t, text}: user's typed answer as a child event. **Patch** (2026-06-11); the old inference still runs as fallback.

Live counts: ask 455, parents 152, internal 139, amend 135, answer 41.

requests/links.jsonl

Writer: backfill (_handle_rep:1102).

- {kind:"link", reply_id, request_ids:[...], relevance, sid, t, did_by_request?:{rid:phrase}}

One reply emits multiple link rows, one per relevance group. relevance is authoritative for routing and may disagree with the reply's own summary tag. did_by_request is a **patch** for the cross-workstream phrase leak.

requests/cleared.jsonl

Writer: the UI only. Read by the folds.

- {id, t}: the one human-asserted fact (Clear). Binary, no done-versus-abandoned distinction.

requests/followups.jsonl

Writer: the UI only. **Patch**, expected to wither.

- {id:<root ask id>, sid, t, text}: reverse-matched to a typed turn by sid, a time window of 15 minutes, and text (_followup_root backfill:460). Violates the event-based rule.

requests/corrections.jsonl

Writer: any session relaying a user verdict.

- {t, by_sid, kind, decision_ref, should_have:{request_ids, relevance}, note}: merges as a read-time link (newest wins); links.jsonl is never rewritten.

requests/decision-log.jsonl

Writer: backfill (_declog:485). Rotated at 2 MB.

- {reply_id|msg_id, sid, t, candidates:[{id,text}], chosen:[...], raw, ...}: a measurement log; the suspect auditor re-reads its own rows. Live kinds: link 731, ask-capture 534, parents 131, req-decision 112, rejudge 85, suspect-audit 46, brief-contradiction 38, tag-override 25, manual-repair 2.

Other records

- decision-brief/<reply_id>.json: writer backfill --brief. {context, question, options|null, needed, sid, t}, prewarmed on needs-user links.
- feed-detail/<eid>.json: writer romp-feed-detail. {id, t, paragraph, relevance, src, next_steps?}, skipped for DETAILS as a cost gate.
- digest/<sid>.json: writer romp-digest. {t, summary, bullets:[{text, t}]}.
- events-cache/<fsid>.json: writer romp-events. {key:[mtime,size,CACHE_VERSION], sid, data:{events, pending, compactions}}.
- names/<sid>: writer bin/romp. Tab-delimited 4 fields, name<TAB>dir<TAB>bg_hex<TAB>fg_color. Outlives the session, so it backs dead-session resolution and revive.
- states/<sid>.jsonl: writer tmux-status.sh, romp-idle-dots, and the headless backends. {t, state}, state in waiting, working, idle, permission, compacting, picker. One line per transition; picker rows add {tier}.
- postal/mail/<sid>/{tmp,new,cur}/: Maildir; the filename is the message id and joins the maildir, timeline/messages.jsonl, and the status-bar prefix. postal/mail-pending/<sid> is a zero-byte marker that exists exactly when unread mail does.
- timeline/messages.jsonl: writer romp-postal. {ev:"sent", id, from, from_id, to_id, body, t, park?} plus exec and recall rows. timeline/message-summaries.jsonl holds {id, summary}.
- Top-level singletons: chat-active, timeline-focus.json, timeline-hover.json, session-order.json, usage.json, web-kernel.json, idle-dots.pid, summarize-backfill.pid, locate-diag.jsonl, REQUESTS.md, SEARCH.md. drops/ holds dropped screenshots and is not pipeline-related.

What the data model shows

- Five node kinds, but answer, parents, and amend carry patch weight; answer shortcuts an inference that still runs as fallback.
- Six relevance tags, three effective. Only DONE and the needs-user group (DECISION/ACTION/IDEA) and "the rest" drive distinct behavior; romp-feed-detail models only three.
- Two id systems (anchor sid and fork fsid) coexist for fork-safety and cost a translation layer everywhere.
- t is vestigial as a key but still stored and used in some time windows.

The fold rules

Enumerated from bin/romp-feed ask_items and mirrored in kernel/feed.ts computeAskItems. LB marks load-bearing core; P marks a patch or rare-case rule. The P rows are the targets for Finding 2.

#	Rule	romp-feed
R1	Node status is the newest non-DETAILS link (DONE to done, else open)	:297
R2	DETAILS never re-opens a verdict (latching)	:296
R3	ACTION is a question only an explicit "did it" closes	:304
R4	DECISION is a question unless answered (user typed later, or session filed newer work)	:307
R5	IDEA is a question dismissed by the next typed turn alone	:310
R6	Rollup: an intermediate node is done iff all paths below are done; a question bubbles up	:350
R7	Column: any open question to needs_input; else all-done and no open follow-up to completed; else asks	:395
R8	Drop point per open leaf (owner is to_sid or sid)	:384
R9	Clear hides the card (newest clear wins)	:236
R10	Resurrection: a cleared card reopens on a later question or follow-up	:378
R11	Follow-up reopens a completed card until minted-as-child or a newer verdict	:368
R12	Corrections merge as read-time links (newest wins)	:259
R13	Answer rows merge as ANSWER links that cross off a question	:273

#	Rule	romp-feed
R14	Child asks render inside their root, never as their own card	:329
R15	Liveness ring: active, delegated, stalled, settled	:407
R16	Auto-filing: a settled card in asks (no follow-up, no WAIT) moves to completed	:428
R17	WAIT exempts a card from auto-filing	:413
R18	Claim-lag hold: a busy owner with an unclaimed open turn holds filing	:423
R19	Blind-read guard: empty states means never mass-auto-file	:419
R20	Conservative claim: a busy owner with an unresolved open turn holds its cards	:407
R21	Corrections are excluded from the done-tally and recency	:432

The core is roughly R1, R3, R4 (first half), R6, R7, R8, R9, R12, R14, R15, R16. The rest are patches, and several presentation rules are re-derived again in webview/feed.ts (group cards, lowest open question, the liveness anomaly).

The model roles

Every call runs `claude -p --tools "" --strict-mcp-config --mcp-config '{}'` with zero tools.

Role	Model	Prompt	Output
Judge: request	Sonnet	REQUEST_SYS :114	PHRASE/ASK/AMEND <n>/ANSWER <n>/ACK
Judge: reply	Sonnet	REPLY_SYS :148 (~65 lines, largest)	TAG :: phrase :: LINK <n> :: DONE <n> :: DID
Judge: message	Sonnet	MSG_SYS :213	phrase :: REQ <yes no> :: PARENTS <n>
Auditor: brief	Opus	BRIEF_SYS :1184	NEEDED/CONTEXT/QUESTION/OPTION
Auditor: suspect	Opus	SUSPECT_SYS :1375	VERDICT :: handoff fyi unsure :: PARENTS ::
Writer: digest	Sonnet	SYS digest:31	SUMMARY: plus – bullet [i]
Writer: feed-detail	Sonnet	SYS feed-detail:45	PARAGRAPH:/NEXT:/TAG:
Announcer	Haiku	in romp-summarize.sh	live @claude-summary tmux var

Volume is about 470 judge calls a day against about 30 auditor calls.

Where code is duplicated

Implemented in	And again in	And a third time
Ask/feed fold: bin/romp-feed	kernel/feed.ts	romp-pipeline_disposition (par
Transcript parse: bin/romp-events	transcript.ts	n/a
Recency colormap: romp_colormap.py	feed.ts feedRamp	render.ts STOPS, timeline, romp-l
Perceptual fade	render.ts:841	timeline:32 and romp-logic.js
MODEL_CHOICES / EFFORT	render.ts:2198	timeline:142
Esc-interrupt heal	romp-idle-dots_heal	romp-interrupt-reset (plus the S
Status-bar reconcile	backfill_push_msg_to_statusbar	backfill_reconcile_statusbars
Client capabilities (openFile/pickFile/clipboard)	extension.ts (VS Code)	browser shim (kernel)
Ruleset as prose	feed.ts help overlay	the code in lvAnomaly/updateAskC

Deletable features, with line counts

Each is removable without touching the spine, so these are the low-risk first cuts. Line counts are rough.

- ☐ **Missed-handoff suspect auditor:** backfill:1368-1496 (~110), plus feed.ts missedHandoffSuspects:584 and the server overlay. "Near-zero precision" per its own comment.
- ☐ **--rejudge:** backfill:1501-1586 (~85). Dead one-time backfill.
- ☐ **headless and stream backends:** headless-backend.ts (265) and stream-backend.ts (262), plus the tui:null branches through server.ts and chat.ts. Keep one or zero.

- ❑ **Postal immediate-wake injection:** `_box_region`, `_is_prompt_box`, `_inject`, `_picker_tier`, `_push`, `romp-postal:621-994` (~370). The drain is the sufficient backstop.
- ❑ **Postal retry/wake/orphan loops:** `_retry_*`, `_wake_when_ready`, `_sweep_orphans` (~100), plus their time constants.
- ❑ **find_sessions / session-search:** `_session_records`, `_find_sessions`, `format_find`, `_resolve_session`, `romp-postal:411-538` (~130). One MCP tool and one CLI verb.
- ❑ **Postal HTTP bus:** justified only by the remote-tunnel feature; a local-only design could use a file lock.
- ❑ **Cosmetic tmux mail decoration:** peer chips, message pill, badge, `romp-postal:641-769` (~130), plus the `status-format[1]` 10-slot string (`bin/romp:153`).
- ❑ **romp-manager multi-kernel registry:** future-proofing for a single kernel (205).
- ❑ **Blocked-session synthetic card:** `feed.ts:1244-1391`. Self-described as impossible.
- ❑ **romp-timeline-serve:** superseded by `kernel /timeline` (66).
- ❑ **Vestigial code:** the `record_summary` no-op stub (`romp-summarize.sh:90`), the retired `romp -g` (`bin/romp:517`), the unreachable `VERBATIM` branch (`romp-summarize:265`), the likely-unused `DEDUP=20` (`backfill:62`), the permanent one-time guards `REQUESTS_FL00R` and `REJUDGE_BY`, and `kernel-attach.ts` (unimported by the kernel).